Sprawozdanie: Warsztat nr 3

Numer zespołu: 32 Igor Skiba (331530)
Termin warsztatu: 16 kwietnia 2026, 16:15–20:00 Radosław Rytel-Kuc (331524)
Link do ostatniego commita w trakcie zajęć: Link
Link do commita z dodaniem pipeline CI/CD: Link

Obserwacje i dokumentacja:

Podczas warsztatu nr 3 zrealizowano proces automatyzacji wdrażania klastra Kubernetes zgodnie z koncepcją *Infrastructure as Code* (IaC) oraz w oparciu o podejście *GitOps*. Prace zostały podzielone na trzy kluczowe etapy:

- **Provisioning (Terraform):** Skonfigurowano środowisko chmurowe OpenStack, definiując maszyny wirtualne w postaci kodu. Uwierzytelnienie z API OpenStack zrealizowano poprzez plik `openrc.sh`. Plik nie został umieszczony w GitLabie. Projekt zorganizowano w czterech plikach: `main.tf` (definicje maszyn i adresów IP), `variables.tf` (deklaracje zmiennych), `terraform.tfvars` (wartości zmiennych) oraz `outputs.tf` (eksport danych). Opracowano pliki konfiguracyjne powołujące jeden węzeł *master*, dynamiczną liczbę węzłów *worker* (zależną od zmiennej `worker_count`). Zautomatyzowano przypisywanie adresów *Floating IP* do wdrożonych instancji. Istotnym elementem integracji było automatyczne generowanie pliku inwentarza dla Ansible (`inventory.ini`) przy pomocy mechanizmu zwracania danych w pliku `outputs.tf`.
- **Configuration management (Ansible):** Narzędzie Ansible wykorzystano do bezagentowej instalacji środowiska Kubernetes na powołanych maszynach. Przygotowano wieloetapowy playbook (`site.yml`), który:
  1. Skonfigurował środowisko bazowe na wszystkich węzłach (m.in. instalacja `containerd`, `kubelet`, `kubeadm`, wyłączenie pamięci `swap`).
  2. Zainicjalizował węzeł *control-plane* (`kubeadm init`), zainstalował wtyczkę sieciową Flannel CNI oraz dynamicznie przechwycił komendę dołączania.
  3. Idempotentnie dołączył pozostałe węzły robocze do utworzonego klastra.

```
ubuntu@k8s-master:~$ kubectl get nodes
NAME         STATUS   ROLES           AGE     VERSION
k8s-master   Ready    control-plane   14m     v1.29.15
worker-1     Ready    <none>          2m13s   v1.29.15
worker-2     Ready    <none>          2m13s   v1.29.15
ubuntu@k8s-master:~$ kubectl get pods -A
NAMESPACE      NAME                                       READY   STATUS    RESTARTS   AGE
kube-flannel   kube-flannel-ds-s5qwq                      1/1     Running   0          15m
kube-flannel   kube-flannel-ds-vhwxw                      1/1     Running   0          3m1s
kube-flannel   kube-flannel-ds-zrf62                      1/1     Running   0          3m1s
kube-system    coredns-76f75df574-nt5pg                   1/1     Running   0          15m
kube-system    coredns-76f75df574-zzhrj                   1/1     Running   0          15m
kube-system    etcd-k8s-master                            1/1     Running   0          15m
kube-system    kube-apiserver-k8s-master                  1/1     Running   0          15m
kube-system    kube-controller-manager-k8s-master         1/1     Running   0          15m
kube-system    kube-proxy-28rdt                           1/1     Running   0          15m
kube-system    kube-proxy-6psjr                           1/1     Running   0          3m1s
kube-system    kube-proxy-tgblw                           1/1     Running   0          3m1s
kube-system    kube-scheduler-k8s-master                  1/1     Running   0          15m
ubuntu@k8s-master:~$
```

Rysunek 1: Weryfikacja klastra - wykonanie komend: kubectl get nodes oraz kubectl get pods -A

Poprawność wdrożenia podstawowej części zweryfikowano na węźle *master* poleceniami `kubectl get nodes` oraz `kubectl get pods -A`. Wszystkie węzły osiągnęły pożądany status `Ready`, a niezbędne pody systemowe zostały pomyślnie uruchomione.

- **Pełne Wdrożenie End-to-End (GitLab CI/CD):** W ramach zadania dodatkowego w pełni zautomatyzowano pipeline'u wdrażania infrastruktury oraz instalacji klastra. W tym celu:
  1. Na stworzonej wcześniej maszynie *Runner* zainstalowano oprogramowanie Docker oraz *GitLab Runner*, a następnie połączono go z uczelnianym GitLabem.
  2. Zabezpieczono dane uwierzytelniające do API OpenStack jako ukryte zmienne środowiskowe, a także skonfigurowano udostępnianie klucza SSH.
  3. Opracowano plik `.gitlab-ci.yml` wykorzystujący obraz `gitlab-terraform` oraz wykonano migrację lokalnego pliku stanu (`.tfstate`) bezpośrednio na serwery GitLaba.
  4. Zdefiniowano czteroetapowy pipeline (`validate`, `plan`, `apply`, `configure`). W zaimplementowanym procesie, zaraz po powołaniu maszyn, adresy są generowane do pliku inwentarza i przesyłane do etapu `configure` z wykorzystaniem mechanizmu *GitLab Artifacts*, zamykając bezobsługowy proces powoływania klastra reagujący na `git push`.

Poprawność wdrożenia dodatkowej części po uruchomieniu pipeline'u w Gitlabie zweryfikowano na węźle `master` poleceniami `kubectl get nodes` oraz `kubectl get pods -A`. Węzły `master` i `worker-1` osiągnęły status `Ready`. Instancja `Runner`, działająca poza klastrem, pozostawała aktywna i podłączona do GitLaba.

Utworzone środowisko stało się w pełni zautomatyzowane i odtwarzalne. W przypadku awarii węzłów lub innego błędu klastra, integracja Terraforma z Ansible pozwala zburzyć infrastrukturę a następnie odbudować ją z poziomu pipeline'u CI/CD za pomocą instancji Runnera działającego jako odrębna maszyna wirtualna.

Warszawa, 24 kwietnia 2026